

Architecture of Massively Scalable Applications, Spring 2026

Milestone 1: Freelance Marketplace
Core Services Foundation
Deadline is Tuesday 19/05/2026 at 11:59 PM

Contents

1 Overview	5
2 Personalization & Team Roster	5
2.1 team.json (Required)	5
2.2 Branch Naming (Mandatory).....	6
2.3 Commit Message Format (Mandatory).....	6
3 Project Structure — Multi-Module Maven	6
3.1 Port & Database Configuration.....	6
4 Step-by-Step: Creating the Multi-Module Project	6
4.1 Using IntelliJ IDEA (Recommended).....	7
4.1.1 A1. Create the Parent Project	7
4.1.2 A2. Add Each Service as a Module.....	7
4.1.3 A3. What IntelliJ Does (and Does Not Do) Automatically	8
4.1.4 A4. Verify and Complete the Root POM	8
4.1.5 A5. Important: Do NOT Modify Child Service POMs.....	9
4.1.6 Add jackson-databind Dependency	9
4.1.7 Configure application.properties per Service.....	9
4.1.8 Create Sub-Packages	9
4.1.9 Add a Health Endpoint to Each Service.....	9
4.1.10 Create team.json	10
4.1.11 Verify the Build	10
4.2 Final Project Structure.....	10
5 Git Workflow — Initial Setup	11
6 Database Setup (Docker Compose)	11
7 Entity Models	11
7.1 User Service Entities (user-service).....	12
7.1.1 User Entity.....	12
7.1.2 UserSkill Entity	12

7.2	Job Service Entities (job-service).....	12
7.2.1	Job Entity	12
7.2.2	JobAttachment Entity	13
7.3	Proposal Service Entities (proposal-service)	13
7.3.1	Proposal Entity	13
7.3.2	ProposalMilestone Entity.....	14
7.4	Contract Service Entities (contract-service).....	14
7.4.1	Contract Entity	14
7.5	Wallet Service Entities (wallet-service).....	15
7.5.1	Payout Entity	15
7.5.2	PromoCode Entity	15
7.5.3	PayoutPromo Entity (Join Entity)	16
7.6	Relationship Summary	16
7.7	Important: Cross-Service Data Access	17
8	Repository Layer	17
9	Features	17
9.1	User Service Features (port 8081)	17
9.1.1	[S1-F1] Search Users with Filters.....	17
9.1.2	[S1-F2] Update User Preferences (JSONB)	18
9.1.3	[S1-F3] Get User Contract Summary (DTO)	18
9.1.4	[S1-F4] Deactivate User Account	18
9.1.5	[S1-F5] Filter Users by Preference (JSONB Query)	19
9.1.6	[S1-F6] Top Freelancers by Earnings (Report DTO)	19
9.1.7	[S1-F7] Set Primary Skill (Transactional + Relationship).....	19
9.1.8	[S1-F8] Get User Profile with Skills (Relationship DTO)	20
9.1.9	[S1-F9] Find Users by Language Preference with Minimum Contracts (JSONB + SQL).....	20
9.2	Job Service Features (port 8082)	20
9.2.1	[S2-F1] Search Jobs by Status and Budget Range.....	20
9.2.2	[S2-F2] Update Job Requirements (JSONB Partial Update).....	21
9.2.3	[S2-F3] Get Job Proposal Summary (DTO).....	21
9.2.4	[S2-F4] Close Job Posting (Transactional)	21
9.2.5	[S2-F5] Filter Jobs by Requirement (JSONB).....	22
9.2.6	[S2-F6] Top Budget Jobs Report (DTO)	22
9.2.7	[S2-F7] Rate Job Client After Contract (Transactional)	22
9.2.8	[S2-F8] Verify Job Attachment (Transactional + Relationship).....	23
9.2.9	[S2-F9] Get Jobs with Expired Attachments (Relationship + Report DTO).....	23
9.3	Proposal Service Features (port 8083).....	23
9.3.1	[S3-F1] Get Proposals by Status and Date Range.....	23
9.3.2	[S3-F2] Accept Proposal and Create Contract (Transactional)	24
9.3.3	[S3-F3] Get Platform Fee Estimate (DTO).....	24

9.3.4	[S3-F4] Complete Proposal's Contract (Transactional Multi-Step)	25
9.3.5	[S3-F5] Filter Proposals by Metadata Field (JSONB)	25
9.3.6	[S3-F6] Proposal Analytics by Time Period (Report DTO)	26
9.3.7	[S3-F7] Withdraw Proposal (Transactional)	26
9.3.8	[S3-F8] Add Milestones to Proposal (Transactional + Relationship)	26
9.3.9	[S3-F9] Get Proposal Details with Milestones (Relationship DTO)	27
9.4	Contract Service Features (port 8084)	27
9.4.1	[S4-F1] Get Active Contract for User	27
9.4.2	[S4-F2] Update Contract Progress with Metadata (JSONB)	27
9.4.3	[S4-F3] Find Contracts by Budget Range with Freelancer Info (DTO)	28
9.4.4	[S4-F4] Batch Contract Status Update (Transactional)	28
9.4.5	[S4-F5] Filter Contracts by Metadata (JSONB Query)	28
9.4.6	[S4-F6] Get Contracts in Date Range	29
9.4.7	[S4-F7] Purge Old Contract Data (Transactional)	29
9.4.8	[S4-F8] Freelancer Performance Summary (DTO)	29
9.4.9	[S4-F9] Find Stalled Contracts (JSONB + Cross-Table DTO)	30
9.5	Wallet Service Features (port 8085)	30
9.5.1	[S5-F1] Get Payouts by Status and Date Range	30
9.5.2	[S5-F2] Process Refund (Transactional + JSONB Update)	30
9.5.3	[S5-F3] Freelancer Payout Summary (DTO)	31
9.5.4	[S5-F4] Process Payout for Contract (Transactional)	31
9.5.5	[S5-F5] Apply PromoCode to Payout (Transactional + Join Entity)	32
9.5.6	[S5-F6] Revenue Report by Date Range (Report DTO)	33
9.5.7	[S5-F7] Retry Failed Payout (Transactional)	34
9.5.8	[S5-F8] Get Payout Details with Applied PromoCodes (Join Entity DTO)	35
9.5.9	[S5-F9] Get Most Used Promo Codes Report (Join Entity + Aggregation)	35
10	Git Workflow — Feature Development	36
10.1	Merge Order (Checkpoint-Gated)	36
11	Dockerization (Phase D)	36
12	Work Distribution (Recommended):	37
13	Submission & Auto-Grader	37
14	Appendix: JSONB Sample Data	37
15	Appendix: Example Database Tables	39
15.1	User Service Tables	39
15.1.1	users table	39
15.1.2	user_skills table	39
15.2	Job Service Tables	40
15.2.1	jobs table	40

15.2.2	job_attachments table.....	41
15.3	Proposal Service Tables	41
15.3.1	proposals table	41
15.3.2	proposal_milestones table.....	41
15.4	Contract Service Tables	42
15.4.1	contracts table	42
15.5	Wallet Service Tables	42
15.5.1	payouts table	42
15.5.2	promo_codes table	43
15.5.3	payout_promos table (Join Entity)	43
15.6	How the Tables Connect (Cross-Service).....	43

1 Overview

This milestone is worth **15%** of your final grade. You will build a **Freelance Marketplace** consisting of 5 independently runnable Spring Boot services organized as a **Maven multi-module project**. All 5 services connect to a single shared PostgreSQL database and are orchestrated via a single docker-compose.yaml.

In future milestones, these services will be refactored into true microservices with separate databases, inter-service communication (OpenFeign, RabbitMQ, etc.), and Kubernetes deployment. For now, they are independent applications sharing one database.

Technologies: Spring Boot, PostgreSQL, Spring Data JPA, Docker, Docker Compose, Maven multi-module, Git & GitHub.

Services:

- a) **User Service** (port 8081) — Freelancer/client accounts, profiles, skills
- b) **Job Service** (port 8082) — Job postings, categories, requirements
- c) **Proposal Service** (port 8083) — Proposals, bidding, milestones
- d) **Contract Service** (port 8084) — Contracts, progress tracking, deliverables
- e) **Wallet Service** (port 8085) — Payment processing, refunds, revenue

Deliverables: 45 features (9 per service), full CRUD for all entities, Dockerized multi-service application, Git history with personalized feature branches and PRs.

2 Personalization & Team Roster

Every team member's work must be **individually identifiable** in Git history. The auto-grader cross-references the team.json roster against git log to determine who built what. Violations result in **zero credit** for the affected member.

2.1 team.json (Required)

Create a file named team.json in the **project root directory** (next to the parent pom.xml). It contains a JSON array with one entry per team member:

```
[{"studentId":"55-8078","name":"Ahmed Ali","githubUsername":"ahmed-ali-dev","service":"user-service"}, ...]
```

Fields per entry:

- studentId — University ID in the format XX-XXXX
- name — Full name
- githubUsername — The exact GitHub username that will author commits
- service — Which service module this member belongs to (one of: user-service, job-service, proposal-service, contract-service, wallet-service)

The auto-grader reads this file and then runs specific tests to find each member's commits. It matches those commits against branch names to determine which features each member implemented. **Do not self-declare features** — the grader discovers them automatically.

2.2 Branch Naming (Mandatory)

All feature branches must include the implementing member's student ID:

`feat/<service>/<feature-name>/<studentId>`

Example: student 55-8078 implementing S1-F1 (Service 1, Feature 1) creates:

`feat/user/S1-F1/55-8078`

2.3 Commit Message Format (Mandatory)

All commits must follow this convention:

`feat(<service name>): <description> (ID)`

Examples: `feat(user-service): add User entity model (55-8078)`, `fix(proposal-service): fix null handling in search query (55-8078)`.

The auto-grader matches each commit's Git author (from `team.json`) against the student ID in the message. Mismatched or missing IDs will result in failures during grading. So even if you named the commit message wrong or the branch name was wrong and did follow the criteria, you know how to fix it using the commands that you took in the Git Lab.

3 Project Structure — Multi-Module Maven

The project is organized as a Maven multi-module project. The root directory contains a **parent POM** with `<packaging>pom</packaging>` that declares all 5 services as modules. Each service is a fully independent Spring Boot application with its own POM, source tree, and `@SpringBootApplication` entry point. Section 4 provides full step-by-step instructions for creating this structure.

3.1 Port & Database Configuration

All services connect to the **same PostgreSQL database** (freelancedb). Each runs on a different port:

Service	Internal Port	Docker Host Port
user-service	8080	8081
job-service	8080	8082
proposal-service	8080	8083
contract-service	8080	8084
wallet-service	8080	8085

Each service's `application.properties` sets `server.port=8080`. The port differentiation happens in `docker-compose.yml` via host-port mapping (e.g., `8081:8080` for `user-service`). When running services locally without Docker during development, each developer works on their own service and can use port 8080 since they only run one service at a time.

Each service configures: `spring.datasource.url=jdbc:postgresql://localhost:5432/freelancedb` (or `postgres:5432` inside Docker), `ddl-auto=update`, and `show-sql=true`.

4 Step-by-Step: Creating the Multi-Module Project

This section walks you through creating the entire project structure from scratch. Two approaches are provided: **Option A** using IntelliJ IDEA (recommended), and **Option B** using Spring Initializr for

students not using IntelliJ. Both produce the same result. Follow **one** of the two options, then continue with the shared steps.

Important architectural note: The root project acts as an **aggregator only**. It does not contain source code and does not act as a Maven parent for the child modules. Each service module keeps its own generated Spring Boot parent. The root pom.xml simply lists the modules so you can build them all with a single command.

4.1 Using IntelliJ IDEA (Recommended)

4.1.1 A1. Create the Parent Project

- a) Open IntelliJ IDEA → **File** → **New** → **Project**.
- b) On the left panel, select **Java**.
- c) On the right, set **Build system** to **Maven**.
- d) Set the **JDK** to **25**.
- e) Uncheck **Add sample code**.
- f) Expand **Advanced Settings** (at the bottom of the dialog).
- g) Fill in:
 - **Name:** freelance
 - **GroupId:** com.teamXX.freelance (replace XX with your team number)
 - **ArtifactId:** freelance
- h) Click **Create**.
- i) IntelliJ creates a Maven project with a pom.xml. Open this pom.xml and manually add the following line inside the <project> block (after <version>):

```
<packaging>pom</packaging>
```
- j) If IntelliJ created a src/ folder in the root project, **delete it** — the root aggregator project has no source code.
- k) **Reload Maven:** Click the Maven reload icon (circular arrows) in the Maven tool window, or right-click the pom.xml → **Maven** → **Reload project**.

4.1.2 A2. Add Each Service as a Module

Repeat the following for each of the 5 services (user-service, job-service, proposal-service, contract-service, wallet-service):

- a) Right-click the project root (freelance) in the Project panel → **New** → **Module**.
- b) Select **Spring Boot** on the left.
- c) Configure the service information:
 - **Name / Artifact:** the service name (e.g., user-service)
 - **Group:** com.teamXX.freelance
 - **Package name:** Use the service domain name **without hyphens**:
 - com.teamXX.freelance.user
 - com.teamXX.freelance.job

- com.teamXX.freelance.proposal
- com.teamXX.freelance.contract
- com.teamXX.freelance.wallet
- **Type / Build system:** Maven
- **Java:** 25
- **Packaging:** Jar

Warning: The artifactId can use hyphens (e.g., user-service), but **Java package names cannot contain hyphens**. Use .user, not .user-service, in the package name.

- Click **Next** to continue to the dependencies page.
- Select these dependencies: **Spring Web, Spring Boot DevTools, Spring Data JPA, PostgreSQL Driver, Rest Repositories**.
- Click **Create**.

4.1.3 A3. What IntelliJ Does (and Does Not Do) Automatically

After creating each service module, IntelliJ will usually:

- Create the service folder with its own pom.xml and src/ tree.
- Generate the @SpringBootApplication main class.

However, IntelliJ may **not** automatically add the <module> entry to the root pom.xml. You must verify this yourself after each module creation.

4.1.4 A4. Verify and Complete the Root POM

After creating all 5 service modules:

- Open the root freelance/pom.xml.
- Confirm that <packaging>pom</packaging> exists.
- Check whether all 5 services appear inside a <modules> block. If **any are missing**, add them manually:

```
<modules>
  <module>user-service</module>
  <module>job-service</module>
  <module>proposal-service</module>
  <module>contract-service</module>
  <module>wallet-service</module>
</modules>
```

- Save the file.
- Reload Maven** to pick up the changes.
- Confirm there are no Maven errors in the tool window.

4.1.5 A5. Important: Do NOT Modify Child Service POMs

Each generated service already has its own <parent> block pointing to Spring Boot. **Do not replace or modify this parent.** The root freelance POM acts only as an **aggregator** (it lists the modules so you can build them all at once). It is **not** a Maven parent for the child modules.

In other words:

- **Do not** add a <parent> block pointing to freelance inside any child POM.
- **Do not** remove the existing Spring Boot <parent> from any child POM.
- The only connection between the root POM and the children is the <modules> block in the root.

You are done with Option A. Skip to *Shared Steps* below.

4.1.6 Add jackson-databind Dependency

In each service's pom.xml, add the following dependency inside the <dependencies> block (required for Hibernate JSONB support):

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

After adding, **reload Maven**.

4.1.7 Configure application.properties per Service

Inside each service's src/main/resources/application.properties, add:

```
server.port=8080

spring.datasource.url=jdbc:postgresql://localhost:5432/freelancedb
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=
    org.hibernate.dialect.PostgreSQLDialect
```

All services use port 8080 internally. The port differentiation to 8081–8085 happens in docker-compose.yml (covered in the Dockerization section).

4.1.8 Create Sub-Packages

Inside each service's main package (e.g., src/main/java/com/teamXX/freelance/user/), create these sub-packages: model/, repository/, service/, controller/, dto/.

4.1.9 Add a Health Endpoint to Each Service

In each service's controller/ package, create a simple health controller that exposes a GET endpoint returning "OK" (200):

- User Service: GET /api/users/health
- Job Service: GET /api/jobs/health

- Proposal Service: GET /api/proposals/health
- Contract Service: GET /api/contracts/health
- Wallet Service: GET /api/payouts/health

4.1.10 Create team.json

In the project root (next to the root pom.xml), create team.json as described in Section 2.

4.1.11 Verify the Build

Start PostgreSQL via Docker:

```
docker compose up -d
```

(This requires the docker-compose.yaml with just the PostgreSQL service — see Section 6.)

Then build all services from the project root:

```
mvn clean package -DskipTests
```

If the build succeeds, you should see BUILD SUCCESS and a JAR in each service's target/ directory.

To test a single service locally:

```
cd user-service
mvn spring-boot:run
```

Then visit <http://localhost:8080/api/users/health> to confirm it responds with "OK".

4.2 Final Project Structure

After completing all steps, your project should look like this:

```
freelance/
+-- pom.xml                      (root aggregator POM, packaging=pom)
+-- team.json                    (team roster)
+-- docker-compose.yaml          (PostgreSQL, later: all services)
+-- .gitignore
+-- user-service/
|   +-- pom.xml                  (own Spring Boot parent, NOT freelance)
|   +-- Dockerfile               (added in Phase D)
|   +-- src/main/java/com/teamXX/freelance/user/
|       +-- UserApplication.java (@SpringBootApplication)
|       +-- model/
|       +-- repository/
|       +-- service/
|       +-- controller/
|       +-- dto/
+-- job-service/
|   +-- (same structure)
+-- proposal-service/
|   +-- (same structure)
+-- contract-service/
|   +-- (same structure)
+-- wallet-service/
    +-- (same structure)
```

5 Git Workflow — Initial Setup

- a) Initialize a Git repository in the project root.
- b) Create a .gitignore excluding: target/, .idea/, *.iml, .env, *.class, .DS_Store.
- c) Create the parent POM, all 5 child POM files, the package structure, and the team.json file.
- d) Make an initial commit: `init: project setup by (Name_ID)` (e.g: `init: project setup by (Mohamed_Ayman_52_8078)`) (note: this is supposed to be the only commit message that will not follow the commit message rule that is stated in this description) (e.g: `init: project setup by (Mohamed_Ayman_52_8078)`).
- e) Create a **private** GitHub repository named `TeamNumber-TeamName-Freelance` (e.g: `40-Random-Freelance`).
- f) Push to remote and rename the default branch to `main`.
- g) **Branch protection:** Enable “*Require a pull request before merging*” on `main`. (The rule may not be enforced on the free version of git so you have to enforce it manually as the grader will check the generated PRs)
- h) Add `A-Nour98` as a collaborator.
- i) **Each member** must contribute to the project through Git using their own GitHub account. The auto-grader verifies all GitHub usernames from `team.json` appear in history of the repo. ***If someone didn't contribute to the project (no-commits for him/her), will be considered as if they didn't work at all in the milestone and will receive a ZERO as a result in the milestone grade.***

6 Database Setup (Docker Compose)

The `docker-compose.yml` in the project root contains a PostgreSQL service with:

- Container name: `freelance-db`
- Port mapping: `5432:5432`
- Environment variables: `user postgres`, `password postgres`, `database freelancedb`
- Named volume `pgdata` for data persistence

In Phase D (Dockerization), you will add all 5 application services to this same file.

Verify: Run `docker compose up -d` (PostgreSQL only at first), start any service locally, and confirm it connects to the database.

7 Entity Models

All entities use auto-generated Long IDs. Some entities include one JSONB column implemented as a `Map<String, Object>` using the Hibernate JSONB annotations learned in Lab 4. Since all services share one database, tables from different services coexist in the same database schema.

Each service defines JPA relationships (`@OneToMany`, `@ManyToOne`, join entities) **between entities within the same service only**. References to entities in other services are stored as plain Long foreign key columns (not JPA-managed relationships), and cross-service data access uses native SQL JOINS on the shared database. Remember to use `@JsonIgnore` on bidirectional relationships to prevent infinite recursion during JSON serialization.

7.1 User Service Entities (user-service)

7.1.1 User Entity

Table: users

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
name	String	not null	
email	String	not null, unique	
password	String	not null	
phone	String	not null, unique	
role	ENUM ^a	not null	"FREELANCER" or "CLIENT" or "ADMIN"
status	ENUM	not null, default ACTIVE	ACTIVE / DEACTIVATED
preferences	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	Set on creation
userSkills	List<UserSkill>	–	Relational Attribute

^asearch about how you would save an enum inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — preferences: Language, notification settings (email/sms booleans), timezone, profile visibility, hourly rate range.

Relationships: *One* User can have *Many* UserSkills and **User** is the inverse side and **UserSkill** is the owner side.

7.1.2 UserSkill Entity

Table: user_skills

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
skillName	String	not null	e.g., "Java", "Graphic Design"
category	String	not null	e.g., "DEVELOPMENT", "DESIGN"
yearsOfExperience	Integer	not null	
proficiencyLevel	ENUM ^a	not null	BEGINNER / INTERMEDIATE / EXPERT
isPrimary	Boolean	default false	
metadata	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
user	User	–	Relational Attribute

^asearch about how you would save an enum **as an ENUM** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — metadata: Certifications list, portfolio URLs, endorsement count, last used date, tools/frameworks.

Relationships: *Many* UserSkills can belong to *One* User and **User** is the inverse side and **UserSkill** is the owner side.

7.2 Job Service Entities (job-service)

7.2.1 Job Entity

Table: jobs

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
clientId	Long	not null	FK reference to users (not JPA-managed)
title	String	not null	
description	String	not null	
category	ENUM ^a	not null	e.g., WEB_DEV, MOBILE, DESIGN, WRITING
status	ENUM	not null	OPEN / IN_PROGRESS / CLOSED
budgetMin	Double	not null	Minimum budget
budgetMax	Double	not null	Maximum budget
rating	Double	default 0.0	Client rating for this job
totalRatings	Integer	default 0	Count for average calculation
requirements	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
jobAttachments	List<JobAttachment>	–	Relational Attribute

^asearch about how you would save an enum **as an ENUM** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — requirements: Required skills list, experience level (JUNIOR, MID, SENIOR), estimated duration (weeks), remote allowed (boolean), preferred timezone.

Relationships: *One* Job has *Many* JobAttachments and **Job** is the inverse side and **JobAttachment** is the owner side.

7.2.2 JobAttachment Entity

Table: job_attachments

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
type	ENUM ^a	not null	BRIEF / MOCKUP / REFERENCE / CONTRACT_TEMPLATE
fileUrl	String	not null	URL or path to file
expiryDate	LocalDate	not null	
verified	Boolean	default false	
metadata	Map<String, Object>	JSONB	See below
uploadedAt	LocalDateTime	not null	
job	Job	–	Relational Attribute

^asearch about how you would save an enum **as an ENUM** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — metadata: File size (KB), file format, version number, verification notes, rejection reason.

Relationships: *Many* JobAttachments belong to *One* Job and **Job** is the inverse side and **JobAttachment** is the owner side.

7.3 Proposal Service Entities (proposal-service)

7.3.1 Proposal Entity

Table: proposals

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
jobId	Long	not null	FK reference (not JPA-managed)
freelancerId	Long	not null	FK reference (not JPA-managed)
coverLetter	String	not null	
bidAmount	Double	not null	Proposed price
estimatedDays	Integer	not null	Proposed delivery time
status	ENUM ^a	not null	See values below
metadata	Map<String, Object>	JSONB	See below
submittedAt	LocalDateTime	not null	
acceptedAt	LocalDateTime	nullable	Set when accepted
proposalMilestones	List<ProposalMilestone>	–	Relational Attribute

^asearch about how you would save an enum **as an ENUM** inside SQL and what are the necessary configurations that you have to do for JPA

Status values: SUBMITTED, SHORTLISTED, ACCEPTED, REJECTED, WITHDRAWN.

JSONB — metadata: Approach summary, relevant experience, tools/technologies proposed, availability start date, portfolio links.

Note: jobId and freelancerId are plain Long columns, **not** JPA @ManyToOne relationships (Job and User live in different services).

Relationships: *One* Proposal can have *Many* ProposalMilestones and **Proposal** is the inverse side while **ProposalMilestone** is the owner side.

7.3.2 ProposalMilestone Entity

Table: proposal_milestones

A proposal can have multiple milestones (e.g., design phase, development phase, testing). Each milestone has an order within the proposal.

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
milestoneOrder	Integer	not null	1, 2, 3...
title	String	not null	
description	String	not null	
amount	Double	not null	Payment for this milestone
status	ENUM ^a	not null	PENDING / IN_PROGRESS / COMPLETED / APPROVED
metadata	Map<String, Object>	JSONB	See below
proposal	Proposal	–	Relational Attribute

^asearch about how you would save an enum **as an ENUM** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — metadata: Deliverables list, acceptance criteria, estimated days, actual completion date, revision count.

Relationships: *Many* ProposalMilestones belong to *One* Proposal and **Proposal** is the inverse side while **ProposalMilestone** is the owner side.

7.4 Contract Service Entities (contract-service)

7.4.1 Contract Entity

Table: contracts

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
jobId	Long	not null	FK reference (not JPA-managed)
freelancerId	Long	not null	FK reference (not JPA-managed)
clientId	Long	not null	FK reference (not JPA-managed)
proposalId	Long	not null	FK reference (not JPA-managed)
agreedAmount	Double	not null	
status	ENUM ^a	not null	ACTIVE / COMPLETED / TERMINATED / DISPUTED
startDate	LocalDateTime	not null	
endDate	LocalDateTime	nullable	
metadata	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	

^asearch about how you would save an enum **as an ENUM** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — metadata: Payment terms (MILESTONE, HOURLY, FIXED), revision limit, NDA signed (boolean), weekly hours expected, progress percentage, last activity date.

Relationships: None within this service. jobId, freelancerId, clientId, and proposalId are plain Long references to other services' tables in the shared database via native SQL.

7.5 Wallet Service Entities (wallet-service)

This service demonstrates the **join entity pattern** for many-to-many relationships with extra columns (like the OrderItem pattern from Lab 4).

7.5.1 Payout Entity

Table: payouts

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
contractId	Long	not null	FK reference (not JPA-managed)
freelancerId	Long	not null	FK reference (not JPA-managed)
amount	Double	not null	
method	ENUM ^a	not null	BANK_TRANSFER / PAYPAL / CRYPTO
status	ENUM	not null	See below
transactionDetails	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
payoutPromos	List<PayoutPromo>	–	Relational Attribute

^asearch about how you would save an enum **as an ENUM** inside SQL and what are the necessary configurations that you have to do for JPA

Status values: PENDING, COMPLETED, FAILED, REFUNDED.

JSONB — transactionDetails: Gateway response, account last four digits, receipt URL, failure reason.

Relationships: *One* Payout can reference *Many* PayoutPromos and *Payout* is the inverse side while *PayoutPromo* is the owner side.

7.5.2 PromoCode Entity

Table: promo_codes

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
code	String	not null, unique	e.g., "FIRSTJOB20"
discountType	ENUM ^a	not null	PERCENTAGE or FIXED
discountValue	Double	not null	e.g., 20.0
maxUses	Integer	not null	
currentUses	Integer	default 0	
expiryDate	LocalDateTime	not null	
active	Boolean	default true	
metadata	Map<String, Object>	JSONB	See below
payoutPromos	List<PayoutPromo>	–	Relational Attribute

^asearch about how you would save an enum **as an ENUM** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — metadata: Eligible job categories, minimum contract amount, terms and conditions, applicable regions.

Relationships: *One* PromoCode can reference *Many* PayoutPromos and **PromoCode** is the inverse side while **PayoutPromo** is the owner side.

7.5.3 PayoutPromo Entity (Join Entity)

Table: payout_promos

This is a **join entity** linking Payout and PromoCode in a many-to-many relationship with extra columns — the same pattern as OrderItem from Lab 4. A payout can use multiple promo codes, and a promo code can be used on multiple payouts.

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
discountApplied	Double	not null	Actual amount deducted
appliedAt	LocalDateTime	not null	When promo code was applied
payout	Payout	–	Relational Attribute
promoCode	PromoCode	–	Relational Attribute

Relationships:

- *Many* PayoutPromos can belong to *One* Payout and **Payout** is the inverse side while **Payout-Promo** is the owner side.
- *Many* PayoutPromos can belong to *One* PromoCode and **PromoCode** is the inverse side while **PayoutPromo** is the owner side.

7.6 Relationship Summary

Service	Relationship	Pattern Tested
User	User ↔ UserSkill	Bidirectional
Job	Job ↔ JobAttachment	Bidirectional
Proposal	Proposal ↔ ProposalMilestone	Bidirectional + ordering ^a
Contract	(no JPA relations)	Cross-service via native SQL
Wallet	Payout ↔ PromoCode	Join entity (PayoutPromo)

^aYou will use the ordering in a specific feature from the feature list

7.7 Important: Cross-Service Data Access

Since all services share one PostgreSQL database, any service can query any table using **native SQL**. For example, the Proposal Service can JOIN proposals with jobs or users in a native @Query. However, each service only defines JPA entity classes and JPA relationships for entities within its *own* module. Cross-table access is always via native SQL, never via JPA relationships. This pattern will be replaced by Micro-services Communications in Milestone 3.

8 Repository Layer

Each service defines a JpaRepository for **each of its entities**. For services with multiple entities, you need multiple repository interfaces. ***Make sure that you make use of the Repository Layer when interacting with the database and not have the queries at the service layer as the auto grader will test having proper layering logic in your code.*** Repositories include:

- **Naming-convention methods** for simple lookups.
- **Custom @Query methods with native SQL** for complex queries, including JOINS against other services' tables in the shared database.
- @Modifying + @Transactional for UPDATE/DELETE queries.

9 Features

CRUD operations (create, read by ID, read all, update, delete) are the **baseline** for **every entity in every service** and are NOT counted as features. This means: User Service needs CRUD for both User and UserSkill, Job Service for both Job and JobAttachment, Proposal Service for both Proposal and ProposalMilestone, Contract Service for Contract, and Wallet Service for Payout, PromoCode, and PayoutPromo. Implement all CRUD first before starting features.

Important Note: Do not forget to apply the Layered architecture (having repository, service and controller) as the test case will test that each feature is implemented according to the layered architecture as explained to you in the Labs.

9.1 User Service Features (port 8081)

9.1.1 [S1-F1] Search Users with Filters

Branch: feat/user/S1-F1/<ID>

Endpoint: GET /api/users/search?name={name}&email={email}&role={role}

Response: List of users matching any non-null filter. Name and email support partial matching (case-insensitive). Return empty list if no matches.

Note: You may use Native SQL query but you will have to find a way to handle null parameters.

Test scenario:

- Create 3 users via CRUD: one with name "Ahmed", role "FREELANCER"; one with name "Sara", role "CLIENT"; one with name "Ahmed Ali", role "FREELANCER".
- GET /api/users/search?name=Ahmed → should return 2 users ("Ahmed" and "Ahmed Ali").
- GET /api/users/search?role=CLIENT → should return only "Sara".
- GET /api/users/search?name=xyz → should return empty list.
- GET /api/users/search?name=ahmed → should still return 2 (case-insensitive).

9.1.2 [S1-F2] Update User Preferences (JSONB)

Branch: feat/user/S1-F2/<ID>

Endpoint: PUT /api/users/{id}/preferences

Request body: JSON object of preference key-value pairs.

Behavior: Find user –**throws 404 if user not found**. Merge incoming preferences into existing map—do not overwrite. The only case that you overwrite the preferences is that if the same keys are already there in the preferences, otherwise if it's a new preference then add it to the current preferences. Return updated user.

Test scenario:

- Create a user with preferences {"language":"en","timezone":"UTC"}.
- PUT /api/users/{id}/preferences with body {"timezone":"Africa/Cairo","hourlyRate":45}.
- Verify response has {"language":"en","timezone":"Africa/Cairo","hourlyRate":45} — “language” kept, “timezone” updated, “hourlyRate” added.
- PUT /api/users/999/preferences → should return 404.

9.1.3 [S1-F3] Get User Contract Summary (DTO)

Branch: feat/user/S1-F3/<ID>

Endpoint: GET /api/users/{id}/contract-summary

Response DTO (UserContractSummaryDTO): userId, name, totalContracts, completedContracts, terminatedContracts, totalEarnings, averageContractValue.

Behavior: Find user –**throws 404 if user not found**. Aggregate data from the contracts table for this user using native SQL JOINS on the shared database. Build and return DTO.¹

Test scenario:

- Create a user. Create 5 contracts for that user (via contract-service CRUD): 3 COMPLETED with agreedAmounts 500, 1000, 1500; 1 TERMINATED; 1 ACTIVE.
- GET /api/users/{id}/contract-summary → should return: totalContracts=5, completedContracts=3, terminatedContracts=1, totalEarnings=3000, averageContractValue=1000.
- Test with a user who has no contracts → all values should be 0.
- Test with non-existent user ID → 404.

9.1.4 [S1-F4] Deactivate User Account

Branch: feat/user/S1-F4/<ID>

Endpoint: PUT /api/users/{id}/deactivate

Behavior (Transactional): Find user (404). Check no active contracts exist for this user in the contracts table (native SQL, status = ACTIVE)—throw 400 if found. Set status to "DEACTIVATED". Withdraw all SUBMITTED proposals for this user (native SQL UPDATE on proposals table, set status to WITHDRAWN). Save.

Test scenario:

- Create a user. Create a contract with status ACTIVE for that user.
- PUT /api/users/{id}/deactivate → should return 400 (active contract exists).
- Update the contract status to COMPLETED.
- PUT /api/users/{id}/deactivate → should succeed. Verify user status is now “DEACTIVATED”.

¹Since you will use native SQL here, you can construct the DTO manually from the Object[] that will be returned from the query

9.1.5 [S1-F5] Filter Users by Preference (JSONB Query)

Branch: feat/user/S1-F5/<ID>

Endpoint: GET /api/users/preferences/search?key={key}&value={value}

Behavior: Validate key/value not blank –**throws 400 if they are**. Return users whose preferences matches key-value pair.

Test scenario:

- Create 3 users: one with preferences {"language":"ar"}, one with {"language":"en"}, one with {"language":"ar"}.
- GET /api/users/preferences/search?key=language&value=ar → should return 2 users.
- GET /api/users/preferences/search?key=language&value=fr → empty list.
- GET /api/users/preferences/search?key=&value=ar → should return 400.

9.1.6 [S1-F6] Top Freelancers by Earnings (Report DTO)

Branch: feat/user/S1-F6/<ID>

Endpoint: GET /api/users/reports/top-freelancers?startDate={date}&endDate={date}&limit={n}

Response DTO (TopFreelancerDTO): userId, name, totalEarnings, contractCount.

Behavior: The feature means show me the top N freelancers who earned the most money on completed contracts in a given time period. Validate dates –**throws 400 if not valid (if start after end)**. Build the query and construct a DTO from the result of the query in the same way as you did in S1-F3.

Test scenario:

- Create 3 users. Create completed contracts with agreedAmounts: User A = 3000 total, User B = 8000 total, User C = 1000 total, all within March 2026.
- GET /api/users/reports/top-freelancers?startDate=2026-03-01&endDate=2026-03-31&limit=2 → should return User B first, then User A.
- Test with dates where no contracts exist → empty list.
- Test with startDate after endDate → 400.

9.1.7 [S1-F7] Set Primary Skill (Transactional + Relationship)

Branch: feat/user/S1-F7/<ID>

Endpoint: PUT /api/users/{userId}/skills/{skillId}/primary

Behavior (Transactional): Find user –**throws 404 if user not found**. Find the UserSkill by its ID –**throws 404 if not found**. Verify the skill belongs to this user –**throws 400 if it doesn't belong to that user**. Set isPrimary = false on all of the user's existing skills. Set isPrimary = true on the target skill. Save via the JPA relationship (cascade). Return the updated user with their skills.

Test scenario:

- Create a user with 3 skills. Set skill #2 as primary.
- PUT /api/users/{userId}/skills/{skillId}/primary → verify skill #3 is now primary and #2 is no longer primary.
- Test with non-existent user → 404.
- Test with a skill ID that belongs to a different user → 400.

9.1.8 [S1-F8] Get User Profile with Skills (Relationship DTO)

Branch: feat/user/S1-F8/<ID>

Endpoint: GET /api/users/{id}/profile

Response DTO (UserProfileDTO): userId, name, email, phone, preferences (JSONB), skills (list of skill objects with skillName, category, yearsOfExperience, proficiencyLevel, isPrimary, metadata), totalSkills (count).

Behavior: Find user with their skills loaded —*throws (404 if user not found)*. Build the DTO from the user entity and its skill collection. The DTO must include the full list of skills, not just IDs.

Test scenario:

- Create a user with preferences and 3 skills (one marked primary).
- GET /api/users/{id}/profile → verify DTO contains user info, preferences, full skill list with all fields, and totalSkills = 3.
- Create a user with no skills → verify skills is empty list and totalSkills = 0.
- Test with non-existent user → 404.

9.1.9 [S1-F9] Find Users by Language Preference with Minimum Contracts (JSONB + SQL)

Branch: feat/user/S1-F9/<ID>

Endpoint: GET /api/users/preferences/language?lang={lang}&minContracts={n}

Behavior: This feature is as if you want to find me all users who speak language x and have completed at least n contracts. Validate lang not blank —*throws 400 if blank*.

Test scenario:

- Create 3 users: User A (language=ar, 5 completed contracts), User B (language=ar, 2 completed contracts), User C (language=en, 10 completed contracts).
- GET /api/users/preferences/language?lang=ar&minContracts=3 → should return only User A.
- GET /api/users/preferences/language?lang=ar&minContracts=1 → should return User A and User B.
- GET /api/users/preferences/language?lang=&minContracts=1 → 400.

9.2 Job Service Features (port 8082)

9.2.1 [S2-F1] Search Jobs by Status and Budget Range

Branch: feat/job/S2-F1/<ID>

Endpoint: GET /api/jobs/search?status={s}&minBudget={b}&maxBudget={b}

Behavior: Validate range (400 if min > max). Native SQL with optional status filter and budget range on budgetMax. Order by budgetMax descending.

Test scenario:

- Create 3 jobs: budgetMax 5000 (OPEN), budgetMax 2000 (IN_PROGRESS), budgetMax 8000 (OPEN).
- GET /api/jobs/search?status=OPEN&minBudget=3000&maxBudget=10000 → should return 2 jobs, ordered 8000 first.
- GET /api/jobs/search?minBudget=1000&maxBudget=3000 → returns the 2000 job (no status filter).
- GET /api/jobs/search?minBudget=10000&maxBudget=1000 → 400.

9.2.2 [S2-F2] Update Job Requirements (JSONB Partial Update)

Branch: feat/job/S2-F2/<ID>

Endpoint: PUT /api/jobs/{id}/requirements

Request body: JSON object of requirement fields to update.

Behavior: Find job –**throws 404 if not found**. Merge incoming fields into existing requirements map, meaning if different attributes were sent in the json body, then add it to the existing one, else then you will update it. Save the updated data and return the updated job record.

Test scenario:

- Create a job with requirements {"experienceLevel":"MID","remote":true,"skills":["Java"]}.
- PUT /api/jobs/{id}/requirements with body {"experienceLevel":"SENIOR","duration":"8 weeks"}.
- Verify response: remote=true (kept), skills=["Java"] (kept), experienceLevel=SENIOR (updated), duration=8 weeks (added).
- Test with non-existent job → 404.

9.2.3 [S2-F3] Get Job Proposal Summary (DTO)

Branch: feat/job/S2-F3/<ID>

Endpoint: GET /api/jobs/{id}/proposal-summary?startDate={date}&endDate={date}

Response DTO (JobProposalSummaryDTO): jobId, title, totalProposals, averageBidAmount, lowestBid, highestBid.

Behavior: Find job (404). Native SQL aggregate on proposals table for this job in date range. Build DTO.

Test scenario:

- Create a job. Create 5 proposals for that job in March with bidAmounts 500, 800, 1200, 600, 900.
- GET /api/jobs/{id}/proposal-summary?startDate=2026-03-01&endDate=2026-03-31 → total-Proposals=5, averageBidAmount=800, lowestBid=500, highestBid=1200.
- Test with date range where no proposals exist → all zeroes.
- Test with non-existent job → 404.

9.2.4 [S2-F4] Close Job Posting (Transactional)

Branch: feat/job/S2-F4/<ID>

Endpoint: PUT /api/jobs/{id}/close

Request body: status.

Behavior (Transactional): Find job –**throws 404 if not found**. If closing, check no active contracts for this job –**throws 400 if there are active contracts**. Update job status to CLOSED. Reject all SUBMITTED proposals for this job (native SQL UPDATE on proposals table, set status to REJECTED). Save and **return status code 200**.

Test scenario:

- Create a job (OPEN). Create an active contract for it.
- PUT /api/jobs/{id}/close with {"status":"CLOSED"} → 400 (active contract).
- Complete the contract. Now try CLOSED again → 200.
- Verify all SUBMITTED proposals for that job are now REJECTED.

9.2.5 [S2-F5] Filter Jobs by Requirement (JSONB)

Branch: feat/job/S2-F5/<ID>

Endpoint: GET /api/jobs/requirements/search?key={key}&value={value}&status={status}

Behavior: Filter jobs by JSONB requirements key-value pair and optional status (the status is not required and means if the job is OPEN, IN_PROGRESS, or CLOSED).

Test scenario:

- Create 3 jobs: one with requirements {"experienceLevel":"SENIOR"} OPEN, one with {"experienceLevel":"JUNIOR"} OPEN, one with {"experienceLevel":"SENIOR"} CLOSED.
- GET /api/jobs/requirements/search?key=experienceLevel&value=SENIOR&status=OPEN → returns 1 job.
- GET /api/jobs/requirements/search?key=experienceLevel&value=SENIOR → returns 2 jobs (no status filter).
- GET /api/jobs/requirements/search?key=experienceLevel&value=LEAD → empty list.

9.2.6 [S2-F6] Top Budget Jobs Report (DTO)

Branch: feat/job/S2-F6/<ID>

Endpoint: GET /api/jobs/reports/top-budget?limit={n}

Response DTO (TopBudgetJobDTO): jobId, title, budgetMax, totalProposals.

Behavior: Get the top budget (n) jobs and return them as TopJobDTO.

Test scenario:

- Create 3 jobs with budgetMax 5000, 3000, 8000. Create some proposals for each.
- GET /api/jobs/reports/top-budget?limit=2 → returns 2 jobs, 8000 first.
- GET /api/jobs/reports/top-budget?limit=10 → returns all 3.

9.2.7 [S2-F7] Rate Job Client After Contract (Transactional)

Branch: feat/job/S2-F7/<ID>

Endpoint: POST /api/jobs/{id}/rate

Request body: contractId and rating (1–5).

Behavior (Transactional): Find job —*throws 404 if not found*. Verify contract exists and references this job and is COMPLETED —*throws 400 if the contract doesn't reference this job or it's not completed or the rate is not between 1 and 5 and throws 404 if the contract is not found*. Recalculate running average. Update job and Save. Return status **code 200**.

Test scenario:

- Create a job (rating=0, totalRatings=0). Create a COMPLETED contract for this job.
- POST /api/jobs/{id}/rate with {"contractId":1,"rating":5} → 200. Verify rating=5.0, totalRatings=1.
- Rate again with a different completed contract, rating=3 → verify rating=4.0, totalRatings=2.
- Test with rating=6 → 400. Test with non-completed contract → 400. Test with contract belonging to another job → 400.

9.2.8 [S2-F8] Verify Job Attachment (Transactional + Relationship)

Branch: feat/job/S2-F8/<ID>

Endpoint: PUT /api/jobs/{jobId}/attachments/{attachmentId}/verify

Request body: *verifiedBy*.

Behavior (Transactional): Find job –*throws 404 if not found*. Find the JobAttachment by ID –*throws 404 if not found* and verify it belongs to this job –*throws 400 if it doesn't belong to the specified job*. Check the attachment is not expired (expiryDate is in the future) –*throws 400 if expired*. Set verified = true. Update the attachment's JSONB metadata to add verifiedAt timestamp and verifiedBy (from request body). In addition, verify the verifiedBy that is the userID being sent in the request body is an actual **Admin** User –(*throw 403 if not*). Return the updated job with all attachments.

Test scenario:

- Create a job. Add a JobAttachment with expiryDate in the future, verified=false.
- PUT /api/jobs/{jobId}/attachments/{attachmentId}/verify with {"verifiedBy":3} → verify attachment is now verified=true and metadata has verifiedAt and verifiedBy.
- Test with expired attachment → 400. Test with non-ADMIN verifier → 403.
- Test with attachment that belongs to different job → 400.

9.2.9 [S2-F9] Get Jobs with Expired Attachments (Relationship + Report DTO)

Branch: feat/job/S2-F9/<ID>

Endpoint: GET /api/jobs/attachments/expired

Response DTO (JobAttachmentAlertDTO): jobId, jobTitle, jobStatus, expiredAttachments (list of attachments where expiryDate < now), expiredCount.

Behavior: Query all jobs that have at least one expired attachment using a native SQL JOIN between jobs and job_attachments. For each matching job, load their expired attachments via the JPA relationship. Build list of DTOs. Only include jobs with expiredCount > 0.

Test scenario:

- Create 3 jobs. Add attachments: Job A has 1 expired + 1 valid, Job B has 2 valid, Job C has 2 expired.
- GET /api/jobs/attachments/expired → returns 2 DTOs (Job A with expiredCount=1, Job C with expiredCount=2). Job B not included.
- Test with no expired attachments in DB → empty list.

9.3 Proposal Service Features (port 8083)

9.3.1 [S3-F1] Get Proposals by Status and Date Range

Branch: feat/proposal/S3-F1/<ID>

Endpoint: GET /api/proposals/search?status={s}&startDate={d}&endDate={d}

Behavior: Query with date range on submittedAt and optional status of the proposal. Order by most recent.

Test scenario:

- Create 5 proposals: 2 ACCEPTED in March, 1 SUBMITTED in March, 2 ACCEPTED in February.
- GET /api/proposals/search?status=ACCEPTED&startDate=2026-03-01&endDate=2026-03-31 → returns 2 proposals, most recent first.
- GET /api/proposals/search?startDate=2026-03-01&endDate=2026-03-31 → returns 3 (no status filter).

9.3.2 [S3-F2] Accept Proposal and Create Contract (Transactional)

Branch: feat/proposal/S3-F2/<ID>

Endpoint: PUT /api/proposals/{proposalId}/accept

Behavior (Transactional): Find proposal (404). Validate proposal status is SUBMITTED or SHORTLISTED (400—can only accept proposals in these states). Verify the freelancer exists and has role FREELANCER in the users table using native SQL (404 if not found, 400 if not a freelancer). Set the proposal's status to ACCEPTED and acceptedAt to now. Update the job's status to IN_PROGRESS (native SQL UPDATE on jobs table). Create a new contract in the contracts table (native SQL INSERT) with status ACTIVE, agreedAmount = proposal's bidAmount, and startDate = now. Save proposal. Return the updated proposal.

Test scenario:

- Create a SUBMITTED proposal with bidAmount=2000. Create an OPEN job. Create a FREELANCER user.
- PUT /api/proposals/{proposalId}/accept → proposal status=ACCEPTED, acceptedAt set, job status=IN_PROGRESS, new ACTIVE contract created with agreedAmount=2000.
- Try accepting again → 400 (proposal not SUBMITTED/SHORTLISTED anymore).
- Try accepting for a non-freelancer user → 400.
- Try with non-existent proposal → 404.

9.3.3 [S3-F3] Get Platform Fee Estimate (DTO)

Branch: feat/proposal/S3-F3/<ID>

Endpoint: POST /api/proposals/estimate

Request body: bidAmount, estimatedDays.

Response DTO (FeeEstimateDTO): bidAmount, platformFee, freelancerPayout, feePercentage, estimatedDailyRate.

Scenario: Before submitting a proposal, a freelancer enters their bid amount and estimated duration and the app shows them the platform fee and their actual payout — just like when Upwork/Fiverr shows the service fee before you submit. This endpoint computes that estimate. It does **not** create a proposal in the database — it only calculates and returns the result.

Behavior — step by step:

- Validate input:** bidAmount must be positive, estimatedDays must be positive. Throw 400 if invalid.
- Determine platform fee percentage (demand-based pricing):** This simulates how freelance platforms adjust fees based on competition. The idea: if many proposals are already submitted for jobs in a similar budget range, competition is high and the platform charges less to attract more freelancers.

Write a **native SQL query** that counts how many proposals in the proposals table are currently active (status IN SUBMITTED, SHORTLISTED) and have a bid amount within $\pm 20\%$ of the requested bidAmount. For example, if bidAmount is 1000, count proposals where bidAmount BETWEEN 800 AND 1200 and status is active.

Based on the count, apply the following fee tiers:

- 0–5 similar proposals → feePercentage = 20% (low competition, higher fee)
- 6–15 similar proposals → feePercentage = 15% (moderate competition)
- More than 15 similar proposals → feePercentage = 10% (high competition, lower fee)

- Calculate platform fee:** platformFee = bidAmount * feePercentage / 100

- d) **Calculate freelancer payout:** $\text{freelancerPayout} = \text{bidAmount} - \text{platformFee}$
- e) **Calculate estimated daily rate:** $\text{estimatedDailyRate} = \text{freelancerPayout} / \text{estimatedDays}$
- f) **Build and return the DTO** containing all five values.

Test scenario:

- a) POST /api/proposals/estimate with bidAmount=1000, estimatedDays=10.
- b) Verify feePercentage=20 (no active proposals in range), platformFee=200, freelancerPayout=800, estimatedDailyRate=80.
- c) Create 10 SUBMITTED proposals with bids between 800–1200. Call estimate again → feePercentage should be 15.
- d) Verify no proposal was created in the database (this endpoint is read-only).

9.3.4 [S3-F4] Complete Proposal's Contract (Transactional Multi-Step)

Branch: feat/proposal/S3-F4/<ID>

Endpoint: PUT /api/proposals/{id}/complete

Behavior (Transactional): Find proposal (404). Validate status ACCEPTED (400). Update to status REJECTED is invalid—this endpoint completes the work. Verify there is an ACTIVE contract for this proposal in the contracts table (native SQL, 400 if not found). Update the contract status to COMPLETED and set endDate (native SQL UPDATE on contracts table). Update the job status to CLOSED (native SQL UPDATE on jobs table). Create payment record (native SQL INSERT into payouts table) with amount = contract's agreedAmount and status PENDING. Save proposal (keep status ACCEPTED). Return.

Test scenario:

- a) Create an ACCEPTED proposal with an ACTIVE contract (agreedAmount=2000) and a job IN_PROGRESS.
- b) PUT /api/proposals/{id}/complete → contract status=COMPLETED with endDate set, job status=CLOSED, a new PENDING payout created with amount=2000.
- c) Try completing again → 400 (no ACTIVE contract found).
- d) Try completing a SUBMITTED proposal → 400.

9.3.5 [S3-F5] Filter Proposals by Metadata Field (JSONB)

Branch: feat/proposal/S3-F5/<ID>

Endpoint: GET /api/proposals/metadata/search?key={key}&value={value}

Behavior: Validate key/value –**throws 400 if not valid**. Return matches.

Test scenario:

- a) Create 3 proposals: one with metadata {"approach":"agile"}, two with {"approach":"waterfall"}.
- b) GET /api/proposals/metadata/search?key=approach&value=agile → returns 1 proposal.
- c) GET /api/proposals/metadata/search?key=&value=x → 400.

9.3.6 [S3-F6] Proposal Analytics by Time Period (Report DTO)

Branch: feat/proposal/S3-F6/<ID>

Endpoint: GET /api/proposals/analytics?startDate={d}&endDate={d}

Response DTO (ProposalAnalyticsDTO): totalProposals, acceptedProposals, rejectedProposals, totalBidValue, averageBid, acceptanceRate.

Behavior: Calculate the required fields for proposals from the start date till the end date. Build the DTO and return it.

Test scenario:

- Create 10 proposals in March: 4 ACCEPTED (bids: 500,1000,1500,2000), 3 REJECTED, 3 SUBMITTED.
- GET /api/proposals/analytics?startDate=2026-03-01&endDate=2026-03-31 → totalProposals=10, acceptedProposals=4, rejectedProposals=3, totalBidValue=sum of all bids, averageBid=average, acceptanceRate=40.0.

9.3.7 [S3-F7] Withdraw Proposal (Transactional)

Branch: feat/proposal/S3-F7/<ID>

Endpoint: PUT /api/proposals/{id}/withdraw

Behavior (Transactional): Find proposal –*throws 404 if not found*. Validate status SUBMITTED/SHORTLISTED –*throws 400 if not valid*. Set WITHDRAWN. If this was the only active proposal for the job and the job is IN_PROGRESS, revert job status to OPEN (native SQL on jobs). Save.

Test scenario:

- Create a SUBMITTED proposal for an OPEN job.
- PUT /api/proposals/{id}/withdraw → proposal status=WITHDRAWN.
- Try withdrawing an ACCEPTED proposal → 400.
- Try withdrawing a REJECTED proposal → 400.

9.3.8 [S3-F8] Add Milestones to Proposal (Transactional + Relationship)

Branch: feat/proposal/S3-F8/<ID>

Endpoint: POST /api/proposals/{proposalId}/milestones

Request body: A list of milestone objects, each with title, description, amount, and metadata.

Behavior (Transactional): Find proposal (404). Validate proposal status is SUBMITTED or SHORTLISTED (400—cannot add milestones to accepted or rejected proposals). Validate each milestone has a title, description, and positive amount (400). Determine the milestoneOrder for each new milestone by continuing from the proposal's current max milestone order (or starting at 1 if no milestones exist). Validate that the sum of all milestone amounts (existing + new) does not exceed the proposal's bidAmount (400). Create ProposalMilestone entities with status = PENDING and add them to the proposal via the JPA @OneToMany relationship. Save the proposal (cascade persists the milestones). Return the updated proposal with all its milestones, ordered by milestoneOrder.

Test scenario:

- Create a SUBMITTED proposal (bidAmount=2000) with no milestones.
- POST /api/proposals/{proposalId}/milestones with 2 milestones (amounts: 800, 700) → verify milestones created with milestoneOrder 1 and 2, status=PENDING.
- Add 1 more milestone (amount=500) → verify milestoneOrder=3. Total milestone amounts = 2000 = bidAmount (OK).

- d) Try adding another milestone (amount=100) → 400 (would exceed bidAmount of 2000).
- e) Try adding milestones to an ACCEPTED proposal → 400.
- f) Try adding a milestone with missing title → 400.

9.3.9 [S3-F9] Get Proposal Details with Milestones (Relationship DTO)

Branch: feat/proposal/S3-F9/<ID>

Endpoint: GET /api/proposals/{proposalId}/details

Response DTO (ProposalDetailsDTO): proposalId, jobId, freelancerId, status, bidAmount, metadata (JSONB), milestones (list of milestone objects ordered by milestoneOrder, each with id, milestoneOrder, title, description, amount, status, metadata), totalMilestones, completedMilestones (count where status = COMPLETED or APPROVED).

Behavior: Find proposal with its milestones loaded —*throws 404 if not found*. Count milestones by status. Build the DTO with the full milestone list ordered by milestoneOrder ascending.

Test scenario:

- a) Create a proposal with 3 milestones: 2 COMPLETED, 1 PENDING.
- b) GET /api/proposals/{proposalId}/details → verify DTO has totalMilestones=3, completedMilestones=2, milestones ordered by milestoneOrder.
- c) Test with proposal that has no milestones → totalMilestones=0, completedMilestones=0, empty milestones list.
- d) Test with non-existent proposal → 404.

9.4 Contract Service Features (port 8084)

9.4.1 [S4-F1] Get Active Contract for User

Branch: feat/contract/S4-F1/<ID>

Endpoint: GET /api/contracts/user/{userId}/active

Behavior: Verify user exists (—*throws 404 if not found*). Query most recent ACTIVE contract for this user (native SQL, ORDER BY createdAt DESC LIMIT 1). 404 if none.

Test scenario:

- a) Create a user. Create 3 contracts for this user with different createdAt timestamps, all ACTIVE.
- b) GET /api/contracts/user/{userId}/active → should return the most recent active contract.
- c) Test with user that has no active contracts → 404.
- d) Test with non-existent user → 404.

9.4.2 [S4-F2] Update Contract Progress with Metadata (JSONB)

Branch: feat/contract/S4-F2/<ID>

Endpoint: PUT /api/contracts/{contractId}/progress

Request body: metadata map with progress updates.

Behavior: Verify contract exists (404). Merge incoming fields into existing metadata map (e.g., update progressPercentage, lastActivityDate). Save. Return.

Test scenario:

- a) Create a contract.

- b) PUT /api/contracts/{contractId}/progress with {"progressPercentage":50,"lastActivityDate":"2026-01-01"} → 200.
- c) Verify the saved contract metadata has the new progress fields.
- d) Test with non-existent contract → 404.

9.4.3 [S4-F3] Find Contracts by Budget Range with Freelancer Info (DTO)

Branch: feat/contract/S4-F3/<ID>

Endpoint: GET /api/contracts/search?minAmount={a}&maxAmount={a}&status={s}

Response DTO (ContractSummaryDTO): contractId, freelancerName, jobTitle, agreedAmount, status, durationDays.

Behavior: Native SQL JOIN contracts with users and jobs. Filter by amount range and optional status. Calculate durationDays from startDate to endDate (or now if still active). Sort by agreedAmount descending. Map to DTOs.

Test scenario:

- a) Create 3 contracts: amounts 1000 (ACTIVE), 3000 (COMPLETED), 5000 (ACTIVE). With corresponding users and jobs.
- b) GET /api/contracts/search?minAmount=2000&maxAmount=6000&status=ACTIVE → returns the 5000 contract only.
- c) GET /api/contracts/search?minAmount=500&maxAmount=4000 → returns 1000 and 3000 (no status filter).

9.4.4 [S4-F4] Batch Contract Status Update (Transactional)

Branch: feat/contract/S4-F4/<ID>

Endpoint: PUT /api/contracts/batch-status

Request body: List of {contractId, status} objects.

Behavior (Transactional): Validate all contracts exist (404). Validate status transitions are valid (400). Update each contract's status and set endDate if completing. Save all. Return 200 with count.

Test scenario:

- a) Create 3 ACTIVE contracts.
- b) PUT /api/contracts/batch-status with 3 updates to COMPLETED → 200 with count=3.
- c) Verify all 3 contracts now have status COMPLETED and endDate set.
- d) Test with non-existent contract ID → 404.
- e) Test with invalid transition (COMPLETED to ACTIVE) → 400.

9.4.5 [S4-F5] Filter Contracts by Metadata (JSONB Query)

Branch: feat/contract/S4-F5/<ID>

Endpoint: GET /api/contracts/metadata/search?key={k}&operator={op}&value={v}

Operators: eq, gt, lt.

Behavior: Validate operator –**throws 400 if not valid**. For gt/lt cast JSONB to numeric. Return matches.

Test scenario:

- a) Create contracts with metadata: progressPercentage=25, progressPercentage=50, progressPercentage=90.

- b) GET /api/contracts/metadata/search?key=progressPercentage&operator=gt&value=40 → returns 2 contracts (50 and 90).
- c) GET /api/contracts/metadata/search?key=progressPercentage&operator=eq&value=25 → returns 1.
- d) GET /api/contracts/metadata/search?key=progressPercentage&operator=xyz&value=50 → 400.

9.4.6 [S4-F6] Get Contracts in Date Range

Branch: feat/contract/S4-F6/<ID>

Endpoint: GET /api/contracts/history?startDate={d}&endDate={d}&status={s}

Behavior: Query all contracts within the date range on createdAt with optional status filter. Order by createdAt ascending.

Test scenario:

- a) Create 5 contracts across different users: 3 in March (2 ACTIVE, 1 COMPLETED), 2 in February.
- b) GET /api/contracts/history?startDate=2026-03-01&endDate=2026-03-31 → returns 3 contracts ordered by createdAt ascending.
- c) GET /api/contracts/history?startDate=2026-03-01&endDate=2026-03-31&status=ACTIVE → returns 2.
- d) Test with date range where no contracts exist → empty list.

9.4.7 [S4-F7] Purge Old Contract Data (Transactional)

Branch: feat/contract/S4-F7/<ID>

Endpoint: DELETE /api/contracts/purge?olderThanDays={n}

Behavior (Transactional): Calculate cutoff (if the request says olderThanDays=30, then cutoff = today minus 30 days). Only delete COMPLETED or TERMINATED contracts (not ACTIVE or DISPUTED). Count then delete via native @Modifying query. Return deleted count.

Test scenario:

- a) Create 10 contracts: 7 COMPLETED with createdAt 60 days ago, 2 ACTIVE with createdAt 60 days ago, 1 COMPLETED with createdAt today.
- b) DELETE /api/contracts/purge?olderThanDays=30 → returns deletedCount=7 (only old COMPLETED ones, not the ACTIVE ones).
- c) Verify 3 contracts remain (2 ACTIVE + 1 recent COMPLETED).

9.4.8 [S4-F8] Freelancer Performance Summary (DTO)

Branch: feat/contract/S4-F8/<ID>

Endpoint: GET /api/contracts/freelancer/{freelancerId}/summary?startDate={d}&endDate={d}

Response DTO (FreelancerPerformanceDTO): freelancerId, totalContracts, averageContractValue, completionRate, averageDurationDays, totalEarnings.

Behavior: Verify freelancer (404). Native SQL aggregate on contracts in date range. Calculate averageDurationDays from EXTRACT(EPOCH FROM (endDate - startDate)) for completed contracts. completionRate = completed / total * 100. Build DTO.

Test scenario:

- a) Create a freelancer. Create 5 contracts in March: 4 COMPLETED (amounts 1000, 1500, 2000, 2500; durations 10, 15, 20, 25 days), 1 TERMINATED.

- b) GET /api/contracts/freelancer/{freelancerId}/summary?startDate=2026-03-01&endDate=2026-03-31 → totalContracts=5, totalEarnings=7000, averageContractValue=1400, completionRate=80.0, averageDurationDays=17.5.
- c) Test with non-existent freelancer → 404.

9.4.9 [S4-F9] Find Stalled Contracts (JSONB + Cross-Table DTO)

Branch: feat/contract/S4-F9/<ID>

Endpoint: GET /api/contracts/stalled?maxProgress={p}&stalledDays={d}

Response DTO (StalledContractDTO): contractId, freelancerName, jobTitle, agreedAmount, progressPercentage, daysSinceLastActivity.

Behavior: Find ACTIVE contracts whose JSONB metadata has a progressPercentage ≤ maxProgress (cast to numeric) and whose lastActivityDate in metadata is more than stalledDays days ago (or createdAt if no lastActivityDate). Use a native SQL query joining contracts with users and jobs to get names. Calculate daysSinceLastActivity. Map results to DTOs.

Test scenario:

- a) Create 3 ACTIVE contracts with users and jobs. Set metadata: Contract A (progressPercentage=10, lastActivityDate=30 days ago), Contract B (progressPercentage=5, lastActivityDate=2 days ago), Contract C (progressPercentage=80, lastActivityDate=30 days ago).
- b) GET /api/contracts/stalled?maxProgress=50&stalledDays=7 → returns Contract A (progress ≤ 50 AND stalled > 7 days). Not B (recent activity). Not C (high progress).
- c) Test with maxProgress=100&stalledDays=1 → returns A and C.

9.5 Wallet Service Features (port 8085)

9.5.1 [S5-F1] Get Payouts by Status and Date Range

Branch: feat/wallet/S5-F1/<ID>

Endpoint: GET /api/payouts/search?status={s}&startDate={d}&endDate={d}

Behavior: Query with optional status and date range on createdAt. Return results with most recent first.

Test scenario:

- a) Create 5 payouts: 2 COMPLETED in March, 1 REFUNDED in March, 2 COMPLETED in February.
- b) GET /api/payouts/search?status=COMPLETED&startDate=2026-03-01&endDate=2026-03-31 → returns 2, most recent first.
- c) GET /api/payouts/search?startDate=2026-03-01&endDate=2026-03-31 → returns 3 (no status filter).

9.5.2 [S5-F2] Process Refund (Transactional + JSONB Update)

Branch: feat/wallet/S5-F2/<ID>

Endpoint: PUT /api/payouts/{id}/refund

Request body: reason (string).

Behavior (Transactional): Find payout –*throws 404 if not found*. Validate status COMPLETED –*throws 400 if not*. Set REFUNDED. Add refundReason and refundedAt to JSONB transactionDetails. Save.

Test scenario:

- a) Create a COMPLETED payout.
- b) PUT /api/payouts/{id}/refund with {"reason":"contract terminated early"} → status=REFUNDED, transactionDetails has refundReason and refundedAt.
- c) Try refunding again → 400 (already REFUNDED).
- d) Try refunding a FAILED payout → 400.

9.5.3 [S5-F3] Freelancer Payout Summary (DTO)

Branch: feat/wallet/S5-F3/<ID>

Endpoint: GET /api/payouts/freelancer/{freelancerId}/summary

Response DTO (FreelancerPayoutSummaryDTO): freelancerId, totalPayouts, totalAmount, methodBreakdown (a Map<String, Double>).

Behavior — step by step:

- a) **Verify user exists:** Check if the user exists –*throws 404 if not found*.
- b) **Query payment data grouped by method:** Write a native SQL query on the payouts table that filters by freelancer_id = :freelancerId and status = 'COMPLETED', then groups the results by the method column. For each group, compute the count and the sum of amount. The query returns multiple rows — one per payment method the freelancer has received.
- c) **Build the method breakdown map:** Loop through the query results (Object[] rows) and populate a Map<String, Double> where each key is the payout method (e.g., "BANK_TRANSFER") and the value is the total amount for that method.
- d) **Calculate totals:** totalPayouts is the sum of all counts across all methods. totalAmount is the sum of all amounts across all methods. You can compute these by iterating over the map, or by running a separate simpler query.
- e) **Build and return the DTO** with freelancerId, totalPayouts, totalAmount, and the methodBreakdown map.

Test scenario:

- a) Create a user. Create 4 COMPLETED payouts: 2 via BANK_TRANSFER (1500, 2000), 1 via PAYPAL (800), 1 via CRYPTO (500).
- b) GET /api/payouts/freelancer/{freelancerId}/summary → totalPayouts=4, totalAmount=4800, methodBreakdown={"BANK_TRANSFER":3500,"PAYPAL":800,"CRYPTO":500}.
- c) Test with non-existent user → 404.

9.5.4 [S5-F4] Process Payout for Contract (Transactional)

Branch: feat/wallet/S5-F4/<ID>

Endpoint: POST /api/payouts/contract/{contractId}

Request body: method, accountLastFour (optional).

Behavior (Transactional): Verify contract exists –*throws 404 if not found* and is COMPLETED –*throws 400 if having different status*. Check no COMPLETED payout exists for this contract –*throws 400 if not with a message “already paid”*. Update the Payout that was created upon contract completion: set its status to COMPLETED, set its method from the request body, populate JSONB transactionDetails with the provided details. Save. Return **status code 201** indicating successful payment.

Test scenario:

- a) Create a COMPLETED contract with agreedAmount=3000.
- b) POST /api/payouts/contract/{contractId} with {"method":"BANK_TRANSFER","accountLastFour":"9876"} → 201. Verify the existing PENDING payout is now COMPLETED with correct method and JSONB populated.
- c) Try paying again for the same contract → 400 ("already paid").
- d) Try paying for an ACTIVE contract → 400.

9.5.5 [S5-F5] Apply PromoCode to Payout (Transactional + Join Entity)

Branch: feat/wallet/S5-F5/<ID>

Endpoint: POST /api/payouts/{payoutId}/promos/{promoCodeId}

No request body — both IDs come from the URL path.

Scenario: A freelancer has a pending payout of 3000 EGP and wants to apply a platform promo code (e.g., reduced service fee). The system validates the promo code, calculates the discount, and creates a PayoutPromo join entity linking the two — the same pattern as OrderItem from Lab 4.

Behavior (Transactional) — step by step:

- a) **Find payout** by payoutId — *throw 404 if not found.*
- b) **Validate payout status:** The payout's status must be PENDING — *throw 400 with message "cannot apply promo code to a completed/cancelled payout" if status is COMPLETED or REFUNDED.*
- c) **Find promo code** by promoCodeId — *throw 404 if not found.*
- d) **Validate promo code is usable:**
 - PromoCode's active field must be true — *throw 400 if not.*
 - PromoCode's expiryDate must be in the future — *throw 400 if expired.*
 - PromoCode's currentUses must be less than maxUses — *throw 400 if limit reached.*
- e) **Check for duplicate:** Query the PayoutPromo records for this payout to see if this promo code has already been applied — *throw 400 with message "promo code already applied" if found.*
- f) **Calculate the discount amount** based on the promo code's discountType:
 - If PERCENTAGE: $\text{discount} = \text{payout.amount} * \text{promoCode.discountValue} / 100$
Example: 3000 EGP payout with a 10% promo code → $3000 * 10 / 100 = 300$ EGP discount.
 - If FIXED: $\text{discount} = \text{promoCode.discountValue}$
Example: 3000 EGP payout with a 200 EGP fixed promo code → 200 EGP discount.

Then **cap the discount:** if the calculated discount exceeds the payout amount, set it to the payout amount (a promo code cannot produce a negative payment).
- g) **Create the join entity:** Create a new PayoutPromo with:
 - payout = the payout entity (via @ManyToOne)
 - promoCode = the promo code entity (via @ManyToOne)
 - discountApplied = the calculated discount from step 6
 - appliedAt = current timestamp
- h) **Update the promo code:** Increment currentUses by 1.
- i) **Save** the PayoutPromo and the updated promo code.
- j) **Return** the updated payout with its list of applied promo codes.

Test scenario:

- a) Create a PENDING payout (amount=3000). Create a promo code: code="FIRSTJOB20", PERCENTAGE, discountValue=10, maxUses=5.
- b) POST /api/payouts/{payoutId}/promos/{promoCodeId} → PayoutPromo created with discountApplied=300. PromoCode currentUses incremented to 1.
- c) Apply same promo code again → 400 (already applied).
- d) Create a FIXED promo code with discountValue=9999. Apply to a 3000 EGP payout → discountApplied should be capped at 3000.
- e) Test with expired promo code → 400. Test with inactive promo code → 400.
- f) Test applying promo code to a COMPLETED payout → 400.

9.5.6 [S5-F6] Revenue Report by Date Range (Report DTO)

Branch: feat/wallet/S5-F6/<ID>

Endpoint: GET /api/payouts/reports/revenue?startDate={d}&endDate={d}

Response DTO (RevenueReportDTO): totalRevenue, totalTransactions, averagePayout, refundedAmount, refundCount.

Scenario: An admin wants to see how much money the platform processed in a given period, and how much was lost to refunds.

Behavior — step by step:

- a) **Validate dates:** Throw 400 if startDate is after endDate.
- b) **Query payments in date range:** Query payouts in date range.
- c) **The five metrics and how to calculate each:**
 - totalRevenue — Sum of amount for all payouts where status = 'COMPLETED'.
 - totalTransactions — Count of payouts where status = 'COMPLETED'.
 - averagePayout — totalRevenue / totalTransactions. Handle division by zero (return 0).
 - refundedAmount — Sum of amount for all payouts where status = 'REFUNDED'.
 - refundCount — Count of payouts where status = 'REFUNDED'.
- d) **Build and return the DTO** with all five values.

Test scenario:

- a) Create payouts in March: 5 COMPLETED (amounts: 1000,1500,2000,2500,3000 = total 10000), 2 REFUNDED (amounts: 800,1200 = total 2000).
- b) GET /api/payouts/reports/revenue?startDate=2026-03-01&endDate=2026-03-31 → totalRevenue=10000, totalTransactions=5, averagePayout=2000, refundedAmount=2000, refundCount=2.
- c) Test with startDate after endDate → 400.

9.5.7 [S5-F7] Retry Failed Payout (Transactional)

Branch: feat/wallet/S5-F7/<ID>

Endpoint: PUT /api/payouts/{id}/retry

No request body — the payout ID comes from the URL path.

Scenario: A payout failed (e.g., bank transfer rejected, network timeout). The freelancer wants to try again. Instead of creating a new payment, the system retries the existing one — marking it as successful and recording that it was a retry.

Behavior (Transactional) — step by step:

- a) **Find payout** by id — *throw 404 if not found*.
- b) **Validate status:** The payout's status must be FAILED. You can only retry a failed payout — *throw 400 if the status is anything else* (PENDING, COMPLETED, or REFUNDED).
- c) **Update status:** Set the payout's status to COMPLETED.
- d) **Update JSONB** transactionDetails: Two fields to update in the map:
 - **retryAttempt** — Read the current value (it may be 0 or a previous number), increment it by 1, and put it back. For example, if this is the first retry: 0 → 1. If it's the second retry: 1 → 2.
 - **gatewayResponse** — Overwrite with the string "approved" (simulating that the payout gateway accepted it this time).

So if the JSONB was:

```
{
  "gatewayResponse": "rejected",
  "accountLastFour": "9876",
  "retryAttempt": 0,
  "failureReason": "insufficient funds"
}
```

After retry it becomes:

```
{
  "gatewayResponse": "approved",
  "accountLastFour": "9876",
  "retryAttempt": 1,
  "failureReason": "insufficient funds"
}
```

- e) **Save** the payout.
- f) **Return** the updated payout entity.

Test scenario:

- a) Create a FAILED payout with transactionDetails {"gatewayResponse": "rejected", "retryAttempt": 0}.
- b) PUT /api/payouts/{id}/retry → status=COMPLETED, retryAttempt=1, gatewayResponse="approved".
- c) Try retrying a COMPLETED payout → 400.
- d) Try retrying a REFUNDED payout → 400.

9.5.8 [S5-F8] Get Payout Details with Applied PromoCodes (Join Entity DTO)

Branch: feat/wallet/S5-F8/<ID>

Endpoint: GET /api/payouts/{payoutId}/details

Response DTO (PayoutDetailsDTO): payoutId, contractId, freelancerId, originalAmount, method, status, transactionDetails (JSONB), appliedPromoCodes (list of objects, each with promoCode, discountType, discountApplied, appliedAt), totalDiscount (sum of all discounts), finalAmount (originalAmount - totalDiscount).

Behavior: Find payout with its PayoutPromo entries loaded —*throws 404 if not found*. For each PayoutPromo, access the linked PromoCode entity to get the promo code and discount type. Calculate totalDiscount and finalAmount. Build DTO.

Test scenario:

- a) Create a payout (amount=3000). Apply 2 promo codes: one with discountApplied=300, one with discountApplied=200.
- b) GET /api/payouts/{payoutId}/details → originalAmount=3000, appliedPromoCodes has 2 entries with promo codes and discount info, totalDiscount=500, finalAmount=2500.
- c) Test with payout that has no promo codes → totalDiscount=0, finalAmount=originalAmount.
- d) Test with non-existent payout → 404.

9.5.9 [S5-F9] Get Most Used Promo Codes Report (Join Entity + Aggregation)

Branch: feat/wallet/S5-F9/<ID>

Endpoint: GET /api/payouts/promos/top-used?limit={n}

Response DTO (PromoCodeUsageDTO): promoCodeId, code, discountType, discountValue, timesUsed, totalDiscountGiven, active, expired.

Scenario: A platform manager wants to see which promo codes are performing best — which ones are used the most and how much discount they've given out in total.

Behavior — step by step:

- a) **Write a native SQL query** that joins the promo_codes table with the payout_promos join table. Group the results by promo code. For each promo code, compute:
 - timesUsed — the promo code's currentUses value.
 - totalDiscountGiven — SUM(payout_promos.discount_applied) for all PayoutPromo rows linked to this promo code.
- b) **Order by** timesUsed descending (most popular promo codes first) and **limit** to n results.
- c) **Map each row to a** PromoCodeUsageDTO: For each result, populate:
 - promoCodeId, code, discountType, discountValue — straight from the promo code columns.
 - timesUsed — from currentUses.
 - totalDiscountGiven — from the SUM computed in the query.
 - active — the promo code's active field.
 - expired — a computed boolean: true if the promo code's expiryDate is before now, false otherwise. Compute this in Java, not in SQL.
- d) **Return** the list of DTOs.

Test scenario:

- a) Create 3 promo codes. Apply promo code A to 5 payouts (total discount=1500), promo code B to 3 payouts (total=600), promo code C to 1 payout (total=100).

- b) GET `/api/payouts/promos/top-used?limit=2` → returns promo code A (timesUsed=5, totalDiscountGiven=1500) then promo code B (timesUsed=3, totalDiscountGiven=600).
- c) Verify “expired” field is true for promo codes past their expiryDate.

10 Git Workflow — Feature Development

Every team member follows this workflow for **every feature**:

- a) Start from main: `git checkout main && git pull origin main`
- b) Create feature branch: `git checkout -b feat/<service>/<feature-ID>/<YOUR-STUDENT-ID>`
- c) Implement: repository methods → service logic → controller endpoint → test via Postman.
- d) Commit: `git commit -m "feat(<service-name>): <description> (<YOUR-STUDENT-ID>)"` (except only the first commit which is the initialization of the project)
- e) Push and create a Pull Request on GitHub.
- f) At least 1 teammate reviews and approves.
- g) **Merge into main using a regular merge commit (“Create a merge commit” on GitHub). Do NOT use squash merge.** A regular merge preserves the branch name in the merge commit message (e.g., Merge pull request #12 from feat/user/S1-F1/55-8078), which the auto-grader uses to verify who implemented which feature.
- h) **Do NOT delete the feature branch after merging.** The auto-grader will verify that each feature has a correctly-named branch containing the student ID. If you delete the branch, the grader will not be able to verify it and this may result in deductions in your grade.

10.1 Merge Order (Checkpoint-Gated)

- a) **Phase A (Week 1):** Parent POM + all 5 modules with **all entities** (including secondary entities like UserSkill, JobAttachment, ProposalMilestone, PromoCode, PayoutPromo), repositories, CRUD for every entity, health endpoints + team.json. → **Run checkpoint A.**
- b) **Phase B (Week 2):** Features F1–F3 per service (15 total). → **Run checkpoint B.**
- c) **Phase C (Week 3):** Features F4–F6 per service (15 total). → **Run checkpoint C.**
- d) **Phase D (Week 4):** Features F7–F9 per service (15 total) + Dockerfiles + final docker-compose.yaml. → **Run checkpoint D.**

11 Dockerization (Phase D)

- a) Create a Dockerfile inside each service module using `eclipse-temurin:25.0.2_10-jdk`. Copy that service’s JAR and expose port 8080.
- b) Update the root `docker-compose.yaml` to include all 5 services alongside PostgreSQL. Each service:
 - Builds from its own Dockerfile (e.g., `build: ./user-service`)
 - Maps its host port to container port 8080 (e.g., `8081:8080`)
 - Overrides `datasource URL` to `jdbc:postgresql://postgres:5432/freelancedb`
 - Depends on the PostgreSQL service
- c) Build: `mvn clean package -DskipTests` from the root.

- d) Run: `docker compose up --build`
- e) Verify all services respond:
 - `localhost:8081/api/users` (User Service)
 - `localhost:8082/api/jobs` (Job Service)
 - `localhost:8083/api/proposals` (Proposal Service)
 - `localhost:8084/api/contracts` (Contract Service)
 - `localhost:8085/api/payouts` (Wallet Service)
- f) Git: branch `feat/docker/<ID>`, commit with ID, PR, merge.

12 Work Distribution (Recommended):

3 members per service × 5 services = 15 members. Each member implements 3 features. Phase A (all entities + CRUD for every entity) is done collectively by the 3 members of each sub-team.

13 Submission & Auto-Grader

- a) Repository must be **private**.
- b) A-Nour98 must be added as a collaborator.
- c) `team.json` must be present and valid at the project root.
- d) `docker compose up` must start PostgreSQL + all 5 services.
- e) Services must respond at `localhost:8081` through `localhost:8085`.
- f) Git history must show feature branches merged via PRs with student IDs.
- g) Commits must follow `feat(<service>): <desc> (<ID>)` format.
- h) Details about the usage of the auto grader and the testing will be sent later

Important: Grading is **per member**. A feature only counts for the member whose GitHub username authored the commits on a branch containing their student ID. Features with mismatched or missing IDs receive zero credit. (The zero will affect both, the team member and the whole team). The only case that the team member will receive a different grade from the rest of the team is if there is no history of commits that shows that this member worked in the milestone (if his/her work was done then the whole team will receive their grade fully and he/she will receive zero. If his/her work was missing from the overall work, the whole team will be deducted for the missing work and he/she will still receive zero). It is the responsibility of the scrum master to make sure that everyone is working and the whole requirements are being delivered, if there is an issue with any member(s) then a form will be sent for complains that you can fill it

Deadline: The Milestone deadline is Tuesday 19/05/2026 at 11:59 PM

Submission: Submission form will be sent later to the scrum masters.

14 Appendix: JSONB Sample Data

Below is one example per service showing what the JSONB columns look like in practice. A full database seeder will be provided separately.

User.preferences:

```
{
  "language": "en",
  "notifications": {"email": true, "sms": false},
  "timezone": "Africa/Cairo",
  "profileVisibility": "PUBLIC",
  "hourlyRateRange": {"min": 300, "max": 600}
}
```

Job.requirements:

```
{
  "requiredSkills": ["Java", "Spring Boot", "PostgreSQL"],
  "experienceLevel": "SENIOR",
  "estimatedDuration": 8,
  "remoteAllowed": true,
  "preferredTimezone": "GMT+2"
}
```

Proposal.metadata:

```
{
  "approachSummary": "Microservices with Spring Boot",
  "relevantExperience": "5 years in similar projects",
  "toolsProposed": ["IntelliJ", "Docker", "GitHub"],
  "availabilityStart": "2026-04-01",
  "portfolioLinks": ["https://portfolio.example.com/project1"]
}
```

Contract.metadata:

```
{
  "paymentTerms": "MILESTONE",
  "revisionLimit": 3,
  "ndaSigned": true,
  "weeklyHoursExpected": 40,
  "progressPercentage": 65,
  "lastActivityDate": "2026-03-15"
}
```

Payout.transactionDetails:

```
{
  "gatewayResponse": "approved",
  "accountLastFour": "9876",
  "receiptUrl": "https://receipts.example.com/abc",
  "failureReason": null
}
```

15 Appendix: Example Database Tables

The following tables show what your database should look like after seeding some test data. Use these as a reference when testing your CRUD operations and features via Postman. All tables coexist in the same freelancedb database.

Note: JSONB columns are shown as formatted JSON code blocks below each table for readability. In the actual database, each block is stored as a single JSONB column value in the corresponding row.

15.1 User Service Tables

15.1.1 users table

id	name	email	password	phone	role	status	created_at
1	Ahmed Hassan	ahmed@mail.com	pass123	+201011111111	FREELANCER	ACTIVE	2026-03-01 10:00
2	Sara Mohamed	sara@mail.com	pass456	+201022222222	CLIENT	ACTIVE	2026-03-02 14:30
3	Admin User	admin@freelance.com	admin789	+201033333333	ADMIN	ACTIVE	2026-03-01 08:00

JSONB column — preferences:

User 1 (Ahmed — Freelancer):

```
{
  "language": "ar",
  "notifications": {"email": true, "sms": false},
  "timezone": "Africa/Cairo",
  "profileVisibility": "public",
  "hourlyRateRange": {"min": 300, "max": 600}
}
```

User 2 (Sara — Client):

```
{
  "language": "en",
  "notifications": {"email": true, "sms": true},
  "timezone": "Africa/Cairo",
  "profileVisibility": "public"
}
```

15.1.2 user_skills table

id	user_id	skill_name	category	yrs	proficiency	is_primary
1	1	Java	DEVELOPMENT	5	EXPERT	true
2	1	Spring Boot	DEVELOPMENT	3	INTERMEDIATE	false
3	1	React	DEVELOPMENT	2	INTERMEDIATE	false

(*yrs = yearsOfExperience*)

JSONB column — metadata:

Skill 1 (Ahmed — Java):

```
{
  "certifications": ["Oracle Certified Professional"],
  "portfolioUrls": ["https://github.com/ahmed-dev"],
  "endorsementCount": 12,
  "lastUsedDate": "2026-03-10",
  "tools": ["IntelliJ IDEA", "Maven", "Docker"]
}
```

Skill 2 (Ahmed — Spring Boot):

```
{
  "certifications": [],
  "portfolioUrls": ["https://github.com/ahmed-dev/spring-projects"],
  "endorsementCount": 7,
  "lastUsedDate": "2026-03-10",
  "tools": ["Spring Data JPA", "Spring Security"]
}
```

15.2 Job Service Tables

15.2.1 jobs table

id	client	title	category	status	min	max	rating	total_r
1	2	E-Commerce Backend	WEB_DEV	CLOSED	5000	15000	4.8	1
2	2	Mobile App UI	DESIGN	OPEN	3000	8000	0.0	0

(*client = clientId*, *min/max = budgetMin/budgetMax*, *total_r = totalRatings*)

JSONB column — requirements:

Job 1 (E-Commerce Backend):

```
{
  "requiredSkills": ["Java", "Spring Boot", "PostgreSQL"],
  "experienceLevel": "MID",
  "estimatedDuration": 8,
  "remoteAllowed": true,
  "preferredTimezone": "Africa/Cairo"
}
```

Job 2 (Mobile App UI):

```
{
  "requiredSkills": ["Figma", "UI/UX", "Mobile Design"],
  "experienceLevel": "JUNIOR",
  "estimatedDuration": 4,
  "remoteAllowed": true,
  "preferredTimezone": null
}
```

15.2.2 job_attachments table

id	job_id	type	file_url	expiry_date	verified
1	1	BRIEF	/docs/ecommerce-brief.pdf	2026-12-31	true
2	1	REFERENCE	/docs/api-spec.pdf	2026-09-30	true
3	2	MOCKUP	/docs/app-mockup.fig	2026-06-30	false

JSONB column — metadata:

Attachment 1 (Brief -- Verified):

```
{
  "fileSize": 245,
  "fileFormat": "PDF",
  "versionNumber": 2,
  "verificationNotes": "Verified by admin on 2026-03-05"
}
```

15.3 Proposal Service Tables

15.3.1 proposals table

id	job_id	freelancer	bid	days	status	submitted_at
1	1	1	10000	45	ACCEPTED	2026-03-05 09:00
2	2	1	5000	20	SUBMITTED	2026-03-18 14:00

(freelancer = freelancerId, bid = bidAmount, days = estimatedDays)

JSONB column — metadata:

Proposal 1 (Ahmed → E-Commerce, Accepted):

```
{
  "approachSummary": "Microservices architecture with Spring Boot",
  "relevantExperience": "Built 3 similar e-commerce backends",
  "toolsProposed": ["Java 25", "Spring Boot 3", "PostgreSQL", "Docker"],
  "availabilityStart": "2026-03-10",
  "portfolioLinks": ["https://github.com/ahmed-dev/ecommerce"]
}
```

15.3.2 proposal_milestones table

id	prop_id	order	title	amount	status
1	1	1	Database Design	2000	COMPLETED
2	1	2	API Development	5000	COMPLETED
3	1	3	Testing & Deployment	3000	COMPLETED

(prop_id = proposal_id, order = milestoneOrder)

JSONB column — metadata:

Milestone 1 (Database Design -- Completed):

```
{
  "deliverables": ["ER diagram", "Migration scripts", "Seed data"],
  "acceptanceCriteria": "All tables created with proper indexes",
  "estimatedDays": 7,
  "actualCompletionDate": "2026-03-17",
  "revisionCount": 1
}
```

Milestone 3 (Testing -- Completed):

```
{
  "deliverables": ["Unit tests", "Integration tests", "Docker Compose"],
  "acceptanceCriteria": "80% code coverage, all endpoints tested",
  "estimatedDays": 14,
  "actualCompletionDate": "2026-03-25",
  "revisionCount": 0
}
```

Reading the data: Ahmed submitted a proposal for Sara's E-Commerce Backend job at 10,000 EGP with 3 milestones (Database Design, API Development, Testing). All three milestones are completed. Ahmed also submitted a second proposal for the Mobile App UI job that's still pending.

15.4 Contract Service Tables

15.4.1 contracts table

id	job	free	client	prop	amount	status	start_date	end_date	created_at
1	1	1	2	1	10000	COMPLETED	2026-03-10	2026-03-25	2026-03-10

(*job = jobId, free = freelancerId, prop = proposalId, amount = agreedAmount*)

JSONB column — metadata:

Contract 1 (Ahmed ↔ Sara, Completed):

```
{
  "paymentTerms": "MILESTONE",
  "revisionLimit": 3,
  "ndaSigned": true,
  "weeklyHoursExpected": 30,
  "progressPercentage": 100,
  "lastActivityDate": "2026-03-25"
}
```

Reading the data: Contract #1 links Ahmed (freelancer) with Sara (client) for the E-Commerce Backend job. The agreed amount is 10,000 EGP with milestone-based payments. The contract is now completed (all milestones delivered) with an NDA signed.

15.5 Wallet Service Tables

15.5.1 payouts table

id	contract_id	freelancer_id	amount	method	status	created_at
1	1	1	10000	BANK_TRANSFER	COMPLETED	2026-03-25 17:50

JSONB column — transactionDetails:

Payout 1 (Ahmed -- Contract Completed, 10K EGP):

```
{
  "gatewayResponse": "approved",
  "accountLastFour": "9876",
  "receiptUrl": "https://pay.example/payout001",
  "failureReason": null
}
```

15.5.2 promo_codes table

id	code	discount_type	value	max	used	expiry_date	active
1	FIRSTJOB20	PERCENTAGE	20.0	50	1	2026-09-30 23:59	true
2	FLAT200	FIXED	200.0	100	0	2026-12-31 23:59	true
3	LAUNCH10	PERCENTAGE	10.0	30	30	2026-02-28 23:59	false

(value = discountValue, max = maxUses, used = currentUses)

JSONB column — metadata:

PromoCode 1 (FIRSTJOB20):

```
{
  "eligibleJobCategories": ["WEB_DEV", "MOBILE", "DESIGN"],
  "minimumContractAmount": 1000,
  "termsAndConditions": "First completed contract only"
}
```

PromoCode 2 (FLAT200):

```
{
  "eligibleJobCategories": ["WEB_DEV"],
  "minimumContractAmount": 5000,
  "applicableRegions": ["Egypt", "UAE"]
}
```

15.5.3 payout_promos table (Join Entity)

id	payout_id	promo_code_id	discount_applied	applied_at
1	1	1	2000.00	2026-03-25 17:55

Reading the data:

- Payout #1 (Ahmed's contract payout, 10,000 EGP) had promo code FIRSTJOB20 applied — 20% of 10,000 = 2,000 EGP platform fee discount.
- PromoCode FIRSTJOB20 has current_uses = 1 because it was used once by Ahmed.
- PromoCode LAUNCH10 is inactive because it reached its max uses (30/30) and its expiry date has passed.

15.6 How the Tables Connect (Cross-Service)

Since all services share one database, you can trace data across tables:

- a) **Sara (user 2, CLIENT)** posted **Job #1** (E-Commerce Backend, 5K–15K EGP budget). **Ahmed (user 1, FREELANCER)** submitted **Proposal #1** with a 10K EGP bid and 3 milestones.
- b) Sara accepted the proposal, creating **Contract #1** (10K EGP agreed). The contract is now COMPLETED.
- c) Ahmed completed all 3 milestones (Database Design, API Development, Testing & Deployment). Upon contract completion, **Payout #1** (10,000 EGP) was created via S3-F4, processed via S5-F4, and had FIRSTJOB20 promo code applied via S5-F5 (saved 2,000 EGP).
- d) Ahmed's **skills** (Java, Spring Boot, React) match the job's **requirements** (Java, Spring Boot, PostgreSQL), which is how he was able to submit a competitive proposal.

- e) A native SQL query in the **Proposal Service** can JOIN proposals with users (freelancer_id), jobs (job_id), contracts (proposal_id), and payouts (contract_id) to build a complete freelancer dashboard DTO.

This is the kind of cross-service data access you will implement using **native SQL JOINS** on the shared database.